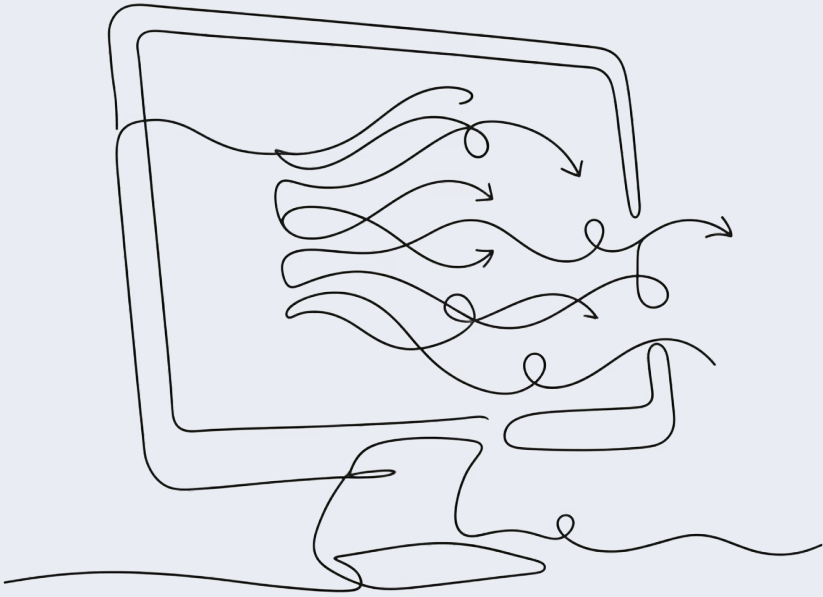




Flujo de Entrada y Salida de Datos en Python

Aprende cómo tu programa puede **hablar con el usuario**, leer archivos y guardar información de forma organizada. Dominar el flujo de E/S es uno de los pasos más importantes para escribir programas útiles y reales.

UNIDAD 2. TEMA 3.1

MSC. ERICKA HERNÁNDEZ MENA

¿Qué es el Flujo de Entrada y Salida?

Todo programa necesita **recibir datos**, procesarlos y **mostrar o guardar resultados**. A esto lo llamamos flujo de Entrada/Salida (E/S o I/O en inglés).



Entrada

El usuario escribe información que el programa recibe y utiliza para operar.



Procesamiento

El programa transforma, calcula o valida los datos recibidos según su lógica.



Salida

El programa muestra resultados en pantalla o los almacena en archivos externos.



Este ciclo — Entrada → Proceso → Salida — es la base de casi todo programa que escribirás.

Entrada de Datos con `input()`

La función `input()` pausa el programa y espera a que el usuario escriba algo. Siempre devuelve una **cadena de texto**, sin importar lo que se escriba.

```
nombre = input("Digite su nombre: ")  
print("Bienvenido", nombre)
```

Cuando necesitas trabajar con números, debes **convertir** el tipo de dato explícitamente:

```
edad = int(input("Digite su edad: "))  
print("El próximo año tendrás", edad + 1)
```

 Recuerda: `int()` convierte a entero y `float()` convierte a decimal. Si el usuario escribe letras cuando esperas un número, el programa lanzará un error.

Conversión de Tipos de Datos

Como `input()` siempre devuelve texto (`str`), necesitas convertir los datos antes de usarlos en cálculos o comparaciones.

A

`str()`

Convierte a cadena de texto. Es el tipo por defecto que devuelve `input()`.

1

`int()`

Convierte a número entero. Útil para edades, cantidades o índices.

10

`float()`

Convierte a número decimal. Ideal para precios, temperaturas o medidas.



`bool()`

Convierte a valor booleano (`True` / `False`). Usado en validaciones.

Formateo de Cadenas

Mostrar información de forma clara es tan importante como obtenerla. Python ofrece varias formas de insertar variables dentro de un texto.

✓ f-string (Recomendado)

La forma más moderna. Pones una `f` antes de las comillas y usas llaves `{}` para insertar variables directamente.

```
nombre = "Ana"
edad = 20
print(f"Soy {nombre} y tengo {edad} años")
```

También puedes formatear números:

```
precio = 15.5678
print(f"Precio: {precio:.2f}")
# Salida: Precio: 15.57
```

↩ Método `.format()`

Forma clásica y compatible con versiones antiguas de Python. Usa llaves vacías como marcadores de posición.

```
nombre = "Luis"
edad = 25
print("Soy {} y tengo {} años"
      .format(nombre, edad))
```

Ambos métodos producen el mismo resultado, pero los f-strings son más legibles y directos.

Escritura de Archivos con `open()`

Python puede crear y modificar archivos de texto usando la función `open()`. Es esencial para guardar información de forma permanente, más allá de la ejecución del programa.

Forma básica

```
archivo = open("datos.txt", "w")
archivo.write("Hola mundo")
archivo.close()
```

✓ Forma recomendada (con `with`)

```
with open("datos.txt", "w") as archivo:
    archivo.write("Hola Python")
```

El bloque `with` cierra el archivo automáticamente al terminar, evitando errores y pérdida de datos.

Modos de apertura

Modo	Descripción
"r"	Lectura (por defecto)
"w"	Escritura (borra contenido previo)
"a"	Agregar al final del archivo
"x"	Crear archivo nuevo
"b"	Modo binario (imágenes, etc.)

Métodos de Objetos de Archivo

Una vez abierto un archivo, dispones de varios métodos para **leer y escribir** su contenido. Cada uno tiene un propósito específico según lo que necesites hacer.



`write()` – Escribir

Escribe una cadena de texto en el archivo.
No añade saltos de línea automáticamente; usa `\n` para eso.

```
archivo.write("Primera línea\n")
```



`read()` – Leer todo

Devuelve todo el contenido del archivo como una sola cadena de texto.

```
contenido = archivo.read()  
print(contenido)
```



`readline()` / `readlines()`

`readline()` lee una línea a la vez.
`readlines()` devuelve una lista con todas las líneas del archivo.

```
lineas = archivo.readlines()  
print(lineas)
```

Guardar Datos Estructurados con JSON

Cuando necesitas guardar información organizada (como diccionarios o listas), el módulo `json` es tu mejor aliado. Es un formato estándar, legible y muy usado en aplicaciones reales.

Guardar en JSON

```
import json

persona = {
    "nombre": "Ana",
    "edad": 20,
    "ciudad": "San José"
}

with open("persona.json", "w") as archivo:
    json.dump(persona, archivo, indent=4)
```

`json.dump()` convierte el diccionario a formato JSON y lo escribe en el archivo. El parámetro `indent=4` hace el archivo más legible.

Leer desde JSON

```
import json

with open("persona.json", "r") as archivo:
    datos = json.load(archivo)

print(datos)
```

`json.load()` lee el archivo y lo convierte de vuelta a un diccionario de Python listo para usar. El resultado sería:

```
{
    "nombre": "Ana",
    "edad": 20,
    "ciudad": "San José"
}
```

Flujo Completo: Un Ejemplo Real

Veamos cómo se unen todas las piezas en un mini-programa que pide datos al usuario y los guarda en un archivo JSON.

```
import json

# 1. Entrada de datos
nombre = input("¿Cuál es tu nombre? ")
edad = int(input("¿Cuántos años tienes? "))

# 2. Formateo y confirmación
print(f"Hola {nombre}, tienes {edad} años.")

# 3. Guardar en archivo JSON
persona = {"nombre": nombre, "edad": edad}

with open("usuario.json", "w") as archivo:
    json.dump(persona, archivo, indent=4)

print("
```

Resumen: Lo que Aprendiste Hoy

Has recorrido todo el ciclo de entrada y salida de datos en Python. Aquí tienes un repaso rápido de los puntos clave:

Entrada con `input()`

Captura texto del usuario. Recuerda convertir con `int()` o `float()` cuando necesites números.

Formateo con f-strings

Usa `f"texto {variable}"` para mostrar datos de forma clara. Es la forma más moderna y recomendada.

Archivos con `open()` y `with`

Escribe y lee archivos de texto. Usa siempre `with` para asegurar que el archivo se cierre correctamente.

Datos estructurados con JSON

Guarda y recupera diccionarios usando `json.dump()` y `json.load()`. Perfecto para datos complejos.

- ❏ ¡Practica creando un programa que guarde una lista de contactos en JSON usando todo lo que aprendiste hoy!

